Chapter 1

Temporal-Difference Learning

Contents

1 Introduction

Temporal-Difference learning (TD) is one of the central ideas in reinforcement learning. Like DP, TD methods update estimates based in part on other learned estimates. On the other hand, TD methods learn directly from experiences like MC methods. DP, MC, and TD methods' relationships will be kept founded in later chapters. n-step algorithms and TD(λ) algorithms are examples of them.

The main advantages of TD over MC and DP methods is as following:

- Do not have to wait until the epsidoes ends.
- Do not have to ignore or discount episodes on which experimental actions are taken.
- No model is required.

2 TD Prediction

TD method's advantage over MC method is that it can be applied to continuing tasks, since it updates each time step. The simplest form of TD method makes the update

$$V(S_t) \leftarrow V(S_t) + \alpha\Big[R_{t+1} + \gamma V(S_{t+1} - V(S_t))\Big] \tag{1.1}$$

immediately on transition to S_t receiving R_{t+1}. This method is called **TD(0)**, and is a speacial case of n-step bootstrap and TD(λ).

2.1 Algorithm

Algorithm 1: Tabular TD(0) Prediction

Input : π ..... policy to be evaluated
$\alpha \in (0, 1]$ ..... step size parameter

Output: v_π ..... state-value function of policy π

```
// initialize
```

foreach $s \in \mathcal{S}$ **do**
  $V(s) \leftarrow \begin{cases} \text{arbitrary value} & \text{if } s \in \mathcal{S}^+ \\ 0 & \text{if } s \in \mathcal{S} \setminus \mathcal{S}^+ \end{cases}$

foreach episodes **do**
  $S \leftarrow$ arbitrary $s \in \mathcal{S}^+$
  foreach step in episode **do**
    $A \leftarrow$ action given by π for S
    take action A, receive reward R and next state S'
    $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
    $S \leftarrow S'$
    if $S \in \mathcal{S} \setminus \mathcal{S}^+$ **then**
      break

return $V \simeq v_\pi$

Since TD(0) method updates value with existing estimates, we say that it **bootstraps**, like DP method. TD combines the sampling of MC and bootstrapping of DP, and we call this update as **sampling update**. Note that it is different from *expected update* of DP.

2.1.1 Convergence of the Algorithm

Since TD(0) method uses estimates instead of complete G_t, one may be skeptical of its convergence. However, it is known to converge to v_π, happily. For any fixed policy π, TD(0) is known to converge to v_π with constant step-size parameter if it is sufficiently small, and is known to converge to v_π with probability 1 if the step-size parameter changes step by step while satisfying the stochastic approximation conditions **??**.

There is no known answer to the question "If TD and MC both converges to correct answer, then which one converges more quickly?". In practice, TD methods usually converge faster than MC methods on stocahstic tasks.

2.2 TD Error

The following definition of *TD error* arises in various forms throughout reinforcement learning:

Definition 2.2.1 (TD Error)

Let V be the estimate of state-value function. Then, the **TD error** δ_t at time t is defined as

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \tag{1.2}$$

Note that TD error is also contained in algorithm 1. If the estimate V does not change during the episode, the Monte Carlo error $G_t - V(S_t)$ can be expressed as the weighted sum of TD errors;

$$G_t - V(S_t) = \sum_{k=t}^{T-1} \gamma^{k-t} \delta_t. \tag{1.3}$$

Even though V changes during episode, the equation above may still hold approximately if the step size is small enough. Generalization of this idea plays an important role in the theory and algorithms of TD learning.

2.3 Batch Updates on TD

We can apply batch updates to TD(0) methods by computing the increments 1.1 every time step but updating the value after the batch, by the sum of all the increments.

Very interesting property can be found on batch TD(0) prediction. Batch TD(0) prediction converges to the **certainty-equivalence estimate** of the MDP. Certainty-equivalence estimate of MDP assumes that the estimate of the MDP is perfect. That is, it is *certain* about its estimate.

To calculate certainty-equivalence estimate directly, it requires memory complexity of $\mathcal{O}(n^2)$ due to the transition probability table, and time complexity of $\mathcal{O}(n^3)$ due to solving the Bellman equation as the linear system of equations. However, using TD(0) method, it requires memory complexity of only $\mathcal{O}(n)$, to save each values of states, which is a surprising result.

Meanwhile, batch MC method converges to minimizer of the mean-square error.

Remark 2.3.1 (TD(0) Converges to Certainty-equivalence Estimate)

uppose that the algorithm converged to fixed point. Then, the next batch update of value of each state s should be 0.

$$\sum_{k\in\text{batch}} \sum_{t\in\{t:S_t^k=s\}} \alpha(R_{t+1}^k + \gamma V(S_{t+1}^k) - V(s)) = 0 \tag{1.4}$$

Let $N(s)$ and $N(s,s')$ denote the number of visits of state s in batch and the number of transition from s to s' in batch, respectively. Then, the equation 1.4 can be written as

$$\left(\sum_{t\in\{t:S_t=s\}} R_{t+1}\right) + \gamma\left(\sum_{s'} N(s,s')V(s')\right) - N(s)V(s) = 0. \tag{1.5}$$

Dividing both side of equation 1.5 with $N(s)$, we get

$$\frac{1}{N(s)}\left(\sum_{t\in\{t:S_t=s\}} R_{t+1}\right) + \gamma\sum_{s'}\frac{N(s,s')}{N(s)}V(s') - V(s) = 0 \tag{1.6}$$

Then, the term $\hat{R}(s) = \frac{1}{N(s)}\left(\sum_{t\in\{t:S_t=s\}} R_{t+1}\right)$ is the average reward of state s and the term $\hat{P}(s'|s) = \frac{N(s,s')}{N(s)}$ is the MLE estimate of transition probability from s to s'. Substituting, we get

$$V(s) = \hat{R}(s) + \gamma\sum_{s'}\hat{P}(s'|s)V(s'), \tag{1.7}$$

which equals the Bellman equation with certainty-equivalence estimate.

3 TD Control

3.1 Sarsa: On-policy TD Control

Sarsa is an on-policy TD control algorithm, where its name rises from the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, which the algorithm uses. The algorithm estimates $q_\pi(s,a)$ instead of $v_\pi(s)$, which can be done using essentially the same TD method described above.

Sarsa uses following update;

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha\Big[R_{t+1} + \gamma Q(S_{t+1},A_{t+1}) - Q(S_t,A_t)\Big] \tag{1.8}$$

This update is done after every transition from a nonterminal state S_t. For terminal state s, $Q(s,a)$ is defined as 0.

Algorithm 2: Sarsa

Input : $\alpha \in (0, 1]$ ..... step size parameter

Output: q_* ..... the optimal policy

```
// initialize
```

foreach $(s, a) \in \mathcal{S} \times \mathcal{A}$ **do**

  $Q(s, a) \leftarrow \begin{cases} \text{arbitrary}, & \text{if } s \in \mathcal{S}^+ \text{ and } a \in \mathcal{A}(s) \\ 0 & \text{if } s \in \mathcal{S} \setminus \mathcal{S}^+ \end{cases}$

foreach episodes **do**

  initialize S

  choose action A from S using policy derived from Q (e.g. ε-greedy)

  foreach step of episode **do**

    take action A, observe R, S'

    choose action A' from S' using policy derived from Q (e.g. ε-greedy)

    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$

    $S \leftarrow S'$

    $A \leftarrow A'$

    if $S \in \mathcal{S} \setminus \mathcal{S}^+$ **then**

      break

return $Q \simeq q_*$

For a Sarsa to converge to optimal policy with probability 1, the policy we use for choosing action must satisfy following property:

- All state-action pairs are visited an infinite number of times;
- Policy converges in the limit to the greedy policy (for example, $\varepsilon = 1/t$)

3.2 Q-learning: Off-policy TD Control

Q-learning is one of the breakthroughs of reinforcement learning. Q-learning uses update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big] \tag{1.9}$$

In this case, the learned action-value function Q directly estimates the optimal action-value function q_*, independent of the policy being followed.

Algorithm 3: Q-learning

Input : $\alpha \in (0, 1]$ ..... step size parameter

Output: q_* ..... the optimal action-value function

```
// initialize
```

foreach $(s, a) \in \mathcal{S} \times \mathcal{A}$ **do**

  $Q(s, a) \leftarrow \begin{cases} \text{arbitrary}, & \text{if } s \in \mathcal{S}^+ \text{ and } a \in \mathcal{A}(s) \\ 0 & \text{if } s \in \mathcal{S} \setminus \mathcal{S}^+ \end{cases}$

foreach episodes **do**

  initialize S

  foreach step of episode **do**

    choose action A from S using behavior policy derived from Q (e.g. ε-greedy)

    take action A, observe R, S'

    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$

    $S \leftarrow S'$

    if $S \in \mathcal{S} \setminus \mathcal{S}^+$ **then**

      break

return $Q \simeq q_*$

For a Q-learning to converge, we do not need additional conditions like Sarsa. What it requires for convergence is to visit and update all state-action pairs, continually.

3.3 Expected Sarsa

Instead of choosing only one action for update like Sarsa or Q-learning, expected Sarsa take into account how likely each action is under the current policy. That is, it takes expectation of $Q(s, a)$ given s.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \mathbb{E}_\pi[Q(S_{t+1}, a)|S_{t+1}] - Q(S_t, A_t)\Big] \tag{1.10}$$

$$= Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\Big] \tag{1.11}$$

Expected Sarsa eliminates the variance due to the random selection of A_{t+1}. It generally learns better than Sarsa.

Note that expected sarsa can be both on-policy and off-policy. If we set π to be greedy and behavior to be exploratory, the expected Sarsa is exactly Q-learning.

Algorithm 4: Expected Sarsa

Input : $\alpha \in (0, 1]$ ..... step size parameter

Output: q_* ..... the optimal action-value function

```
// initialize
foreach (s, a) ∈ S × A do
    Q(s, a) ← { arbitrary,   if s ∈ S⁺ and a ∈ A(s)
              { 0            if s ∈ S \ S⁺
foreach episodes do
    initialize S
    foreach step of episode do
        choose action A from S using behavior policy derived from Q (e.g. ε-greedy)
        take action A, observe R, S′
        get desired target policy π (e.g. greedy policy, ε-greedy policy with decreasing ε) from current Q
        Q(S, A) ← Q(S, A) + α[R + γ Σ_a π(a|S′)Q(S′, a) − Q(S, A)]
        S ← S′
        if S ∈ S \ S⁺ then
            break
return Q ≃ q_*
```

3.4 Double Learning

3.4.1 Maximization Bias

All the control algorithms discussed so far all contains maximization operation. For example, Q-learning obviously has max operation, and Sarsa contains maximization in its ε-greedy policy. This maximizaton often suffers from problem called **maximization bias**.

For example, consider a single state s where there are many actions a with zero values, but estimation are uncertain and distribute above and below zero. The maximum of true values are zero, but the maximum of estimates is positive, a positive bias. This causes slow learning, or even stops it. Note that $\mathbb{E}[\max_a Q(s, a)] \geq \max_a \mathbb{E}[Q(s, a)] = q_*(s, a)$. This is the positive bias.

3.4.2 Double Q-Learning

To avoid this, we use two learners. Suppose that we estimate action-value function q_* with two estimators, Q_1 and Q_2. Then, we use one estimate, say Q_1, to determine the maximizing action $A^* = \arg\max_a Q_1(A, a)$, and use the other, Q_2, to provide the estimate of its value, $Q_2(A^*) = Q_2(\arg\max_a Q_1(a))$. This estimate, is unbiased in the sense that $\mathbb{E}[Q_2(A^*)] = q(A^*)$. We repeat this process by keep changing the role of two estimators.

Algorithm 5: Double Q-learning

Input : $\alpha \in (0, 1]$ ..... step size parameter

Output: q_* ..... optimal action-value function

// initialize

for $i \in \{1, 2\}$ **do**

  foreach $(s, a) \in \mathcal{S} \times \mathcal{A}$ **do**

    $Q_i(s, a) \leftarrow \begin{cases} \text{arbitrary}, & \text{if } s \in \mathcal{S}^+ \text{ and } a \in \mathcal{A}(s) \\ 0 & \text{if } s \in \mathcal{S} \setminus \mathcal{S}^+ \end{cases}$

foreach episodes **do**

  initialize S

  foreach step of episode **do**

    choose action A from S using behavior policy derived from $Q_1 + Q_2$ (e.g. ε-greedy)

    take action A, observe R, S'

    $\begin{cases} \text{with probability 0.5} & Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\big[R + \gamma Q_2(S', \arg\max_a Q_1(S', a)) - Q_1(S, A)\big] \\ \text{else} & Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\big[R + \gamma Q_1(S', \arg\max_a Q_2(S', a)) - Q_2(S, A)\big] \end{cases}$

    $S \leftarrow S'$

    if $S \in \mathcal{S} \setminus \mathcal{S}^+$ **then**

      break

return $Q_1 \simeq Q_2 \simeq q_*$

Note that the algorithm alternates between Q_1 and Q_2.

Remark 3.4.1 (Generalization of Double Learning to other Algorithms)

This mechanism can be applied to any other algorithms. For example, Double Expected Sarsa, we use following update:

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \mathbb{E}_{\pi_1}[Q_2(S_{t+1}, a)|S_{t+1}] - Q_1(S_t, A_t)\Big] \tag{1.12}$$

$$= Q_1(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \sum_a \pi_1(a|S_{t+1}) Q_2(S_{t+1}, a) - Q_1(S_t, A_t)\Big] \tag{1.13}$$

where π_1 is the target policy derived from Q_1. The update for Q_2 is symmetric, interchanging subscript 1 and 2.

The point is to separate the choice of the action and computation of the value of chosen action.